

CIRCUITSYNTH: Leveraging Large Language Models for Circuit Topology Synthesis

Prashanth Vijayaraghavan*
IBM Research
San Jose, CA 95120
prashanthv@ibm.com

Luyao Shi*
IBM Research
San Jose, CA 95120
luyao.shi@ibm.com

Ehsan Degan
IBM Research
San Jose, CA 95120
edehgha@us.ibm.com

Xin Zhang
IBM Research
Yorktown Heights, NY 10598
xzhang@us.ibm.com

Abstract—Circuit topology generation plays a crucial role in the design of electronic circuits, influencing the fundamental functionality of the circuit. In this paper, we introduce CIRCUITSYNTH, a novel approach that harnesses LLMs to facilitate the automated synthesis of valid circuit topologies. With a dataset comprising both valid and invalid circuit configurations, CIRCUITSYNTH employs a sophisticated two-phase methodology, comprising Circuit Topology Generation and Circuit Topology Refinement. Experimental results demonstrate the effectiveness of CIRCUITSYNTH compared to various fine-tuned LLM variants. Our approach lays the foundation for future research aimed at enhancing circuit efficiency and specifying output voltage, thus enabling the automated generation of circuit topologies with improved performance and adherence to design requirements.

Index Terms—LLMs, circuit topology, circuit generation, circuit validity, circuit topology synthesis, language models, netlist, circuit design, power converter

I. INTRODUCTION

Circuit topology synthesis stands as a complex and critical aspect of electronic circuit design. The configuration and interconnection of components directly influence critical circuit functionality and performance. With the increasing demands for integration and complexity in modern electronic systems, the role of circuit topology synthesis becomes crucial in meeting design specifications and performance criteria. However, relying solely on human intervention for topology synthesis is a formidable challenge. As the complexity of contemporary circuit designs grows, the search space expands exponentially, rendering exhaustive or random exploration impractical.

Traditional methods, encompassing rule-based systems, heuristic approaches, and genetic algorithms, have been proposed in the past to tackle circuit topology synthesis. Yet, these methods often encounter limitations in scalability, adaptability to evolving design requirements, and efficiency. While there are different approaches proposed for circuit design process, using Large Language Models (LLMs) for circuit topology synthesis remains relatively underexplored. While LLMs have showcased exceptional capabilities in natural language understanding and generation [1], their application in circuit synthesis remains largely untapped. The ability of LLMs to learn complex patterns, comprehend relationships,

and generate diverse outputs holds promise for overcoming the limitations of traditional methods.

In this context, we introduce CIRCUITSYNTH, an innovative approach that harnesses the power of LLMs for automated circuit topology synthesis. Figure 1 provides an overview of the proposed method, with the goal of generating a valid circuit topology in the form of a netlist based on a text prompt. CIRCUITSYNTH employs a sophisticated methodology, leveraging an extensive dataset of valid and invalid circuit configurations.

Our proposed approach adopts a two-phase model architecture, comprising Circuit Topology Generation and Circuit Topology Refinement. In the initial phase, we fine-tune a LLM to produce a circuit topology in an autoregressive manner. Since the generated circuits may not consistently meet validity constraints, in the Circuit Topology Refinement phase we undertake two pivotal steps. Firstly, we employ a classifier to gauge the likelihood that a generated circuit topology adheres to the requisites of a valid circuit design. This ensures alignment with the necessary design parameters. Secondly, in Generation Enhancement we refine the circuit topology generation process, by minimizing the combined loss of negative log-likelihood with the circuit invalidity score. We evaluate our model by generating new circuit topologies using the trained model and passing them to the SPICE simulator to verify their validity. Experimental results demonstrate the effectiveness of CIRCUITSYNTH compared to various LLM variants, emphasizing its potential for automating circuit topology synthesis.

Contributions: The key contributions are summarized as follows: (a) Introduction of CIRCUITSYNTH, a novel methodology leveraging LLMs for automated circuit topology synthesis; (b) Generation of a comprehensive dataset encompassing both valid and invalid circuit configurations, facilitating the development and evaluation of CIRCUITSYNTH; (c) Development of a circuit validity classifier to assess a circuit's validity, enhancing the reliability of CIRCUITSYNTH outputs.

II. OUR APPROACH

A. Dataset

We curated a dataset of power converter circuit designs with 5 device components using Random Search (RS) [2] and NGSpice [3] simulator. Together we collected a total of 862,606 circuits (567,307 valid and 295,299 invalid ones). In

*These authors contributed equally to this work

this study we only consider devices including capacitors C , inductors L , phase-I switches S_a and phase-II switches S_b . More details about our dataset are described in Appendix A.

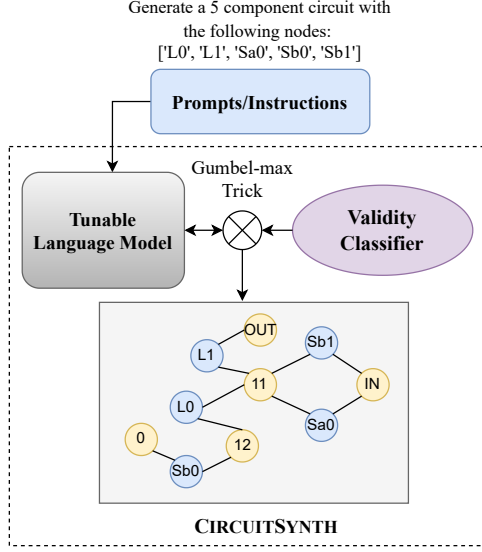


Fig. 1. An overview of our proposed CIRCUITSYNTH. Given a natural language prompt with the component pool, our model can explore the design space effectively and generate circuit topologies by leveraging LLMs.

B. Model Overview

Figure 1 illustrates the model architecture, comprising two main phases: Circuit Topology Generation and Circuit Topology Refinement. In the first phase, we utilize a large language model (LLM) to generate a probability distribution p_{LLM}^t over tokens at each time step t , given an input prompt $X = [x^{(1)}, \dots, x^{(L)}]$ of sequence length L containing a component pool. From this distribution, we sample the circuit topology $Y = [y^{(1)}, \dots, y^{(t)}, \dots, y^{(N)}]$ in an autoregressive manner, where N denotes the sequence length. It should be noted that the generated circuit may not always adhere to validity constraints. To address this, the second phase, Circuit Topology Refinement, consists of two steps:

1) *Circuit Validity Estimation*: Here, we implement a classifier that assesses the probability p_{valid} that the generated circuit topology, represented as a netlist, is valid. This classifier evaluates the compliance of the generated topology with the requirements of a valid circuit design.

2) *Generation Enhancement*: This phase enhances the LLM for valid circuit generation by minimizing the combined loss of negative log-likelihood with the circuit invalidity score. During this stage, the circuit validity classifier remains frozen to ensure efficient training.

Estimating circuit validity involves drawing discrete samples using a categorical distribution from the language model, which poses challenges for gradient-based optimization. To overcome this, we employ the Gumbel softmax trick [4], [5]. Gumbel-Softmax relaxation between the topology generator and the circuit validity classifier enables the backpropagation

of gradients among discrete circuit topology samples during training. We describe these methods in details in the subsequent sections.

C. Circuit Topology Generation

We formulate the task of circuit topology generation as a text generation problem. In this paper, we work with datasets $\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^T$ where X_i refers to the input prompt and Y_i refers to the valid circuit topology netlist. Each entry in the netlist corresponds to a node in an undirected graph $\mathcal{G}$, with edges denoting interconnections between these nodes. The choice of encoding strategy for representing the netlist textually plays a pivotal role, as it significantly influences the effectiveness of LLMs in our task. Therefore, we adopt the ‘‘Incident’’ encoding strategy, known for its superiority over other encoding methods in various graph-related tasks [6]. Figure B.1 in Appendix B demonstrates an example of how a netlist is represented using the incident encoding method.

Formally, the circuit topology generation process can be described as:

$$Y \sim \mathbb{E}_{X \sim \mathcal{D}} [p_{\text{LLM}}(Y|X)] \quad (1)$$

Where p_{LLM} is the pretrained LLM parameterized by a set of parameters θ . This can be further decomposed into:

$$p_{\text{LLM}}(Y|X) = \prod_{t=1}^N p_{\text{LLM}}(y_t|X, \{y_j\}_{j=1}^{t-1}) \quad (2)$$

To tune the pretrained LLM, we use the conventional negative log-likelihood objective:

$$\mathcal{L}_{\text{LLM}} = \sum_{t=1}^N -\log p_{\text{LLM}}(y_t|X, \{y_j\}_{j=1}^{t-1}) \quad (3)$$

This objective generally ensures fluency of text in the incident encoding method. However, this objective can be incomplete. The circuit topology generated may not include all the components in the component pool or satisfy circuit validity constraints. In order to ensure the circuits meet additional constraints, we introduce the circuit topology refinement phase that learns to incorporate specific constraints into the circuit topology generation process.

D. Circuit Topology Refinement

This phase comprises two main steps: (a) Circuit Validity Estimation, which evaluates whether the generated circuit topology adheres to necessary constraints, and (b) Generation Enhancement, which utilizes feedback from the constraint estimation to improve the overall circuit topology generation process. In this study, the primary focus is on optimizing for circuit validity. We elaborate on these steps below.

1) *Circuit Validity Estimation*: We train a dedicated classifier, f_{valid} , on the dataset $\mathcal{D} = \{\mathcal{D}_{\text{valid}} \cup \mathcal{D}_{\text{invalid}}\}$, which includes both valid and invalid circuit topologies. The output of the classifier, p_{valid} , represents the probability of validity for a given circuit topology. We employ a RoBERTa-based classifier [7] optimized with binary cross-entropy loss, achieving an F_1 score of 92% for binary classification of circuit validity.

2) *Generation Enhancement*: To refine the circuit topology generation process, we utilize the pretrained circuit validity classifier to compute a circuit validity loss:

$$\mathcal{L}_{\text{valid}} = (1 - p_{\text{valid}}) \quad (4)$$

Subsequently, we combine this circuit validity loss with the standard negative log-likelihood loss $\mathcal{L}_{\text{LLM}}$ to ensure adherence to circuit validity constraints. Formally, the combined loss function is defined as:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{LLM}} + \lambda_2 \mathcal{L}_{\text{valid}} \quad (5)$$

Here, λ_1 and λ_2 are learnable coefficients that determine the relative importance of each loss component.

During training, a significant challenge arises when sampling $\hat{Y}$ from the distribution p_{LLM} to feed into the circuit validity classifier. This sampling process involves drawing samples from a non-differentiable categorical distribution, hindering gradient propagation [8]. To address this issue, we employ the Gumbel-softmax relaxation [4], [5] to approximate the discrete sampling process $\hat{y}^{(t)} \sim p_{\text{LLM}}$. Firstly, we apply the Gumbel-Max trick to reparameterize sampling from p_{LLM} . This is given as follows:

$$u^{(i)} \sim \text{uniform}(0, 1) \quad (6)$$

$$z^{(i,t)} = -\log(-\log(u^{(i)})) \quad (7)$$

$$\hat{y}^{(t)} = \text{one-hot} \left[\text{argmax}_{i \in |V|} (\hat{p}_{\text{LLM}}^{(i,t)} + z^{(i,t)}) \right] \quad (8)$$

Where $|V|$ is the size of the vocabulary, $\hat{p}_{\text{LLM}}^{(i,t)}$ refers to the logits, i.e., pre-softmax activation of p_{LLM} at the t -th generation step for the i -th word, and $z^{(i,t)}$ are i.i.d. samples from the standard Gumbel distribution. Next, we approximate the discrete *argmax* operation in Equation 8 with the continuous softmax operator to ensure differentiability as:

$$\tilde{y}^{(t)} = \text{softmax} \left(\frac{\hat{p}_{\text{LLM}}^{(i,t)} + z^{(i,t)}}{\tau} \right) \quad (9)$$

where τ is a temperature hyperparameter, which controls how close $\tilde{y}^{(t)}$ is to $\hat{y}^{(t)}$. Finally, to enable gradient flow during training, we utilize the straight-through estimator [9]. In this approach, we use $\hat{y}^{(t)}$ in the forward pass and $\tilde{y}^{(t)}$ in the backward pass, allowing for efficient backpropagation.

III. EXPERIMENTS

In this section, we provide details about the models, baselines and metrics used to train and evaluate. We trained and compared two LLMs for CIRCUITSYNTH: GPT-Neo-2.7 and StableLM-3B-4E1T (referred to as GPT-Neo and StableLM in subsequent sections). More model descriptions and implementation details are provided in Appendix C and Appendix D, respectively.

A. Baselines

We conducted experiments with the following baselines:

1) *Zero-Shot Generation*: We provide prompts containing the components pool as input to LLMs including Llama-2 (13b) [10] and Flan-UI2 (20b) [11] without any fine-tuning.

2) *In-Context Learning (ICL)*: We provide demonstrations of input prompts with component pools and example circuits preceding a new input prompt to Llama-2 (13b) and Flan-UI2 (20b) models. We limit the number of in-context examples to $k \in \{5, 10, 20\}$ and experiment with incident encoding and netlist array-like structures for representing circuit topologies.

3) *Parameter-Efficient Fine-Tuning (PEFT)*: We adopt simple PEFT-based approaches [12] to tune Llama-2 (13b) and Flan-UI2 (20b) models to investigate if a relatively smaller language model (GPT-Neo/StableLM) trained using our approach can achieve comparable performance to much larger LLMs PEFT-tuned for the same task. We performed Prompt-tuning which involves learning task-specific soft prompts and adding them to the input while keeping the pre-trained model parameters frozen. In our study, we explore different numbers of trainable soft prompt tokens, ranging from 100 to 500.

4) *Vanilla Fine-Tuning*: GPT-Neo and StableLM models are fine-tuned with the objective of minimizing the negative log-likelihood for generating circuit topologies.

5) *CIRCUITSYNTH (CS)*: We introduce CIRCUITSYNTH, our complete framework aimed at enhancing fine-tuned circuit topology generation using a circuit validity classifier to improve the validity of generated circuit topologies.

B. Metrics

In our evaluation setup, we report the fraction of unique circuit topologies that are estimated as valid by the independent validity classifier and the SPICE simulator as $E(f_{\text{valid}}(\hat{y}))$ and $E(f_{\text{S_{valid}}}(\hat{y}))$ respectively, based on 1000 samples of unique circuit topologies. In our experiments, a circuit topology is considered valid if it has a validity score, surpassing 0.6, from the circuit validity classifier. Additionally, we report the efficiency of the generated circuit as computed using the SPICE simulator as $E(f_{\text{S_{eff}}}(\hat{y}))$. The SPICE simulator evaluates the validity and efficiency of a given netlist by verifying its electrical characteristics and performance metrics through detailed circuit simulations. It evaluates parameters such as voltage levels, timing, and power consumption of the circuit topology with certain duty cycles under given conditions. Furthermore, we calculate a Duplicate Generation Rate (DGR), denoted by ρ , defined as “# topologies generated” divided by “# unique topologies”.

IV. RESULTS & DISCUSSION

A. Overview

Table I presents the evaluation results of our circuit synthesis models, showing the ratios of valid, unique, and efficient circuits, along with the DGR (ρ). Each model is assessed by generating 1000 unique sample circuit topologies. Our findings suggest that CIRCUITSYNTH models consistently outperforms most baselines across various evaluation metrics. Note that the smaller language models tuned using our method perform comparably with larger prompt-tuned models.

TABLE I
EVALUATION RESULTS. LLAMA AND FLAN REFERS TO LLAMA-2 (13B) AND FLAN-UL2 (20B) MODELS RESPECTIVELY. ρ DENOTES THE DGR METRIC. CS REFERS TO OUR CIRCUITSYNTH MODEL.

Models	$E(f_{\text{valid}}(\hat{y}))$	$E(f_{\text{S}_{\text{valid}}}(\hat{y}))$	$E(f_{\text{S}_{\text{eff}}}(\hat{y}))$	ρ
PEFT Generation				
Llama _{p500}	0.526	0.337	0.611	4.45
Llama _{p100}	0.581	0.648	0.576	3.93
Flan _{p100}	0.608	0.663	0.694	3.65
Fine-tuned Generation				
GPT-Neo _{ft}	0.591	0.60	0.692	1.89
StableLM _{ft}	0.598	0.591	0.682	1.85
CS _{GPT-Neo}	0.636	0.648	0.713	1.31
CS _{StableLM}	0.632	0.624	0.728	1.31

B. Performance of Zero-Shot and ICL Methods

We observe a significant performance gap between zero-shot generation methods and fine-tuned approaches. Zero-shot generation struggles to produce valid netlist-like structures for subsequent classification or simulation. Even with ICL, where the model is provided with examples, there is only a marginal improvement and the completion rate for all components remains unsatisfactory. Furthermore, increasing the number of in-context examples k resulted in diminishing returns.

C. Effectiveness of CIRCUITSYNTH

1) *Comparison with Similar-sized Model Fine-tuning:* Our CIRCUITSYNTH models demonstrate a marked superiority in terms of generating valid circuits compared to vanilla fine-tuned GPT-Neo and StableLM architectures. Specifically, the CIRCUITSYNTH model utilizing GPT-Neo demonstrates a notable enhancement in circuit validity and efficiency compared to its vanilla counterpart. As both models have a similar size, this observation underscores the efficacy of our approach in improving the synthesis of valid circuit topologies. Also, the duplicate generation rate for our CIRCUITSYNTH models is significantly lower compared to other fine-tuned models, indicating faster production of unique circuit topologies.

2) *Comparison with PEFT-tuned Models:* Our experiments demonstrate that PEFT-based prompt tuning of a Flan-ul2 model (~ 20 -billion parameters), yields performances comparable to that of our models with ~ 3 -billion parameters. Notably, we find that a smaller language model (GPT-Neo/StableLM) trained using our approach can achieve comparable performance to LLMs fine-tuned for the same task. The duplicate generation rate of our approach is much lower than the prompt-tuned Llama-2/Flan-ul2 models, which indicates that our method empowers a smaller language model to produce unique circuit topologies faster than PEFT-tuned 20 billion parameter models. An intriguing finding is the capacity of a smaller fine-tuned model to effectively capture extensive information from a larger prompt-tuned model. This highlights the effectiveness of our proposed methodology.

TABLE II
ABLATION STUDY: EFFECT OF NATURAL LANGUAGE (NL) INCIDENT ENCODING FOR TUNING THE MODEL.

Models	$E(f_{\text{S}_{\text{valid}}}(\hat{y}))$	$E(f_{\text{S}_{\text{eff}}}(\hat{y}))$
CIRCUITSYNTH _{GPT-Neo}	0.648	0.713
w/o NL	0.628	0.677
CIRCUITSYNTH _{StableLM}	0.624	0.728
w/o NL	0.640	0.685

D. Validity Correlation

To evaluate the correlation between classifier prediction probabilities and simulator-assessed ground truth validity scores, we used a two-sample t-test. We analyzed 1000 sample circuit topology generations each from CS_{GPT-Neo} and CS_{StableLM}. The classifier prediction probabilities were treated as the continuous predictor variable, while the binary ground truth validity served as the outcome variable. With a statistically significant $p < 0.05$, we found a strong correlation between the classifier predictions and the ground truth validity scores provided by the simulator.

E. Ablation Study

We present an ablation study to investigate the impact of natural language incident encoding on the model's performance by replacing the encoding with an array-like structure to represent netlists (see the left part of Figure B.1). Table II shows the results of our evaluation of CS_{GPT-Neo} and CS_{StableLM} with and without natural language incident encoding. For CS_{GPT-Neo}, we observe significant improvement in both metrics with natural language incident encoding. Conversely, for CS_{StableLM}, the impact is less pronounced. We hypothesize that this disparity is due to differences in their training data. StableLM, trained on both natural language and coding datasets, may rely less on explicit natural language representations for effective circuit synthesis. Thus, our ablation study shows that the benefits of natural language incident encoding depend on the model architecture.

F. Potential Emergent Capability: Efficiency Scores

An emergent capability of our models is the production of circuit topologies with good efficiency scores. Despite the primary objective being the generation of valid circuits, our models demonstrate the additional ability to optimize for efficiency, further enhancing their practical utility.

V. CONCLUSION

In this study, we introduced CIRCUITSYNTH, a novel method that harnesses LLMs to automate circuit topology synthesis. Leveraging a circuit validity classifier and the Gumbel-softmax trick, CIRCUITSYNTH was trained to enhance the overall validity of generated circuits. Experimental results demonstrate that CIRCUITSYNTH yields significant improvements across multiple metrics when compared to various LLM variants. CIRCUITSYNTH offers a promising avenue for revolutionizing electronic circuit design, promising advancements in efficiency, performance, and scalability.

REFERENCES

- [1] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [2] S. Fan, N. Cao, S. Zhang, J. Li, X. Guo, and X. Zhang, “From specification to topology: Automatic power converter design via reinforcement learning,” in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–9.
- [3] P. Nenzi and H. Vogt, “Ngspice users manual version 23,” *Experiments/ngspice23-manual.pdf*, 2011.
- [4] C. J. Maddison, A. Mnih, and Y. W. Teh, “The concrete distribution: A continuous relaxation of discrete random variables,” *arXiv preprint arXiv:1611.00712*, 2016.
- [5] E. Jang, S. Gu, and B. Poole, “Categorical reparameterization with gumbel-softmax,” *arXiv preprint arXiv:1611.01144*, 2016.
- [6] B. Fatemi, J. Halcrow, and B. Perozzi, “Talk like a graph: Encoding graphs for large language models,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=luXR1CCrSi>
- [7] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [8] W. Nie, N. Narodytska, and A. Patel, “Relgan: Relational generative adversarial networks for text generation,” in *International conference on learning representations*, 2018.
- [9] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv preprint arXiv:1308.3432*, 2013.
- [10] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [11] Y. Tay, M. Dehghani, V. Q. Tran, X. Garcia, J. Wei, X. Wang, H. W. Chung, S. Shakeri, D. Bahri, T. Schuster *et al.*, “UI2: Unifying language learning paradigms,” *arXiv preprint arXiv:2205.05131*, 2022.
- [12] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for nlp,” in *International conference on machine learning*. PMLR, 2019, pp. 2790–2799.
- [13] S. Black, L. Gao, P. Wang, C. Leahy, and S. Biderman, “Gpt-neo: Large scale autoregressive language modeling with mesh-tensorflow,” *If you use this software, please cite it using these metadata*, vol. 58, p. 2, 2021.
- [14] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2017.

APPENDIX A DATASET

We curated a dataset of power converter circuit designs with 5 device components. We first used Random Search (RS) [2] to generate numerous random circuit topologies. Then we utilized an open-source electronic circuit simulator NGSpice [3] to identify valid and invalid circuits and collected efficiency values for the valid circuits. Together we collected a total of 862,606 circuits, with 567,307 valid ones and 295,299 invalid ones. In this study we only consider devices including capacitors C , inductors L , phase-I switches S_a and phase-II switches S_b . Each device component has two ports. Together with the three external ports V_{in} , V_{out} and Gnd (renamed with ‘IN’, ‘OUT’ and ‘0’, respectively), the design space contains topologies with 13 ports in total. The device ports are indexed by numbers and the connected ports are represented by only one of the port numbers (see example in Figure. 1). We used fixed device parameters for capacitors ($10\mu F$) and inductors ($100\mu H$). For external ports, we use an input resistor of 0.1Ω for V_{in} , and an output resistor of 50Ω and

an output capacitor of $10\mu F$ for V_{out} . The duty cycle is randomly selected from a set of [0.1, 0.3, 0.5, 0.7, 0.9]. The frequency and input voltage are configured as $1MHz$ and $100V$, respectively.

APPENDIX B INCIDENT ENCODING: EXAMPLE

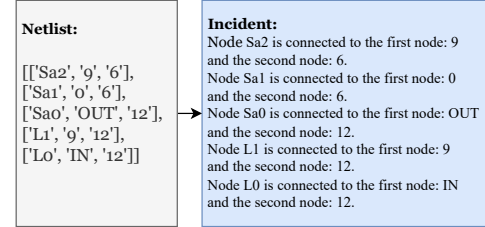


Fig. B.1. Example: incident encoding of a netlist.

APPENDIX C LLM MODELS

More detailed descriptions of the LLMs used for our circuit topology generation task are provided below:

- GPT-Neo-2.7 [13] ¹: GPT-Neo 2.7B is a transformer model designed using EleutherAI’s replication of the GPT-3 architecture. GPT-Neo refers to the class of models, while 2.7B represents the number of parameters of this particular pre-trained model.
- StableLM-3B-4E1T ²: StableLM-3B-4E1T is a 3 billion parameter decoder-only language model pre-trained on 1 trillion tokens of diverse English and code datasets for 4 epochs. We refer to this as StableLM in subsequent sections.

APPENDIX D IMPLEMENTATION DETAILS

We provide the implementation details of our experiments conducted with the official PyTorch v2.2.0 release binary package, compiled with CUDA 11.8, utilizing NVIDIA V100 GPUs with 32 GB of memory.

- Training: We utilize shuffled data from the training split for 5-7 epochs, saving the model checkpoint with the best performance on the validation split. To conserve memory, we implement gradient checkpointing. Additionally, we employ the AdamW optimizer [14] with beta parameters set to 0.9 and 0.95 respectively, and an epsilon value of $1.0E-8$. Moreover, we set the learning rate to $0.95E-5$ and fix the seed to 42 for reproducibility purposes.
- Inference: We assess all models by generating 1000 unique sample circuit topologies using each model. These instances are generated through a combination of nucleus sampling and top-k sampling techniques. Subsequently, the generated circuit topologies are inputted into the

¹<https://huggingface.co/EleutherAI/gpt-neo-2.7B>

²<https://huggingface.co/stabilityai/stablelm-3b-4e1t>

validity classifier to determine the percentage of valid generations. During training, we evaluate a subset of 100 sample generations and employ identical evaluation settings to monitor performance, saving the checkpoint if the current performance surpasses that of the previous epoch.